

22.ONNX模型编辑

部分模型在转RKNN模型后，可能存在冗余的op，常见于模型的输入输出节点处存在冗余的reshape、

transpose op，影响了RKNN模型的推理性能。RKNN-Toolkit2提供了 onnx_edit 接口，用于修改ONNX模型的输入输出的维度定义，使调整后的onnx模型能转出性能更好的RKNN模型，减少冗余的reshape、transpose op。

1. onnx_edit接口说明

使用示例：

代码块

```
1  from rknn.utils import onnx_edit
2  ret = onnx_edit(model = './concat_block.onnx',
3                  export_path = './concat_block_edited.onnx',
4                  inputs_transform = {'k_in': 'a,b,c,d->1,ad,b,c'},
5                  outputs_transform = {'k_cache': 'a,b,c,d->1,ab,c,d'},
6                  dataset = './dataset.txt'
7                  )
```

- model: 填入待修改模型，为必填参数
- export_path: 填入新模型的生成路径，为必填参数
- inputs_transform: 填入输入节点的变换公式字典，key 为节点名称，value 为变换公式。为可选参数，默认为空字典
- outputs_transform: 填入输出节点的变换公式字典。为可选参数，默认为空字典
- dataset: 填入输入数据的路径集文件，文件格式要求与 rknn.build 接口对 dataset 的要求一致。填入后，

onnx_edit 接口除了对模型的输入输出定义进行调整，也会将 dataset 中对应的数据进行调整，在 export_path 同级目录下生成新的 dataset 数据，可用于新模型的验证、量化。为可选参数，默认为空。

2. onnx_edit变换公式说明

onnx_edit接口中的 inputs_transform、outputs_transform 需要填入变换公式。变换公式的定义与 einsum 算子的定义类似，例如 'a,b,c,d->1,ad,b,c' 公式，指将原始的维度为 'a,b,c,d'，变换后为 '1,ad,b,c'

1,ad,b,c'。变换公式的填写规则如下：

- 必须有两部分字符组成，并用 '-' 隔开，左边为原始字符shape，右边是变换后字符shape
- 字符shape里面的符号只允许是 [a-z] 或 ',' '1'，除了'1'以外，不支持其他数字shape字符，原因是相同数字字符无法判断 transpose 的前后关系
- 字符[a-z]认为是独立的，没有顺序关系，即 'a,b->b,a' 和 'c,a->a,c' 表示的是一样的变换
- 等式左边的原始字符shape，被 ',' 分隔成 n 份，n的个数必须和模型的维度匹配，例如模型输入定义是 [32,4,1,64]，输入字符可以是 'a,b,c,d' 或 'a,b,1,d'
- 原始字符shape的每一个符号，除了'1'，都必须存在于变换后字符shape，例如 'a,1,c,d->ac,d,1' 是有效的，'a,1,c,d->ac,1' 是无效的，缺了'd'
- 变换后字符shape，允许插入任意个数的'1'，达成扩维效果，例如 'a,b,c->a,1,cb,1,1'
- 原始字符shape，允许用多个字母以及赋值公式来表示对shape进行拆分，例如原始输入定义是 [32,4,1,64]，'ab,c,d,qk[a=2,k=8]->aq,cd,1,kb'，表示将 32 拆分成 2x16，将 64 拆分成 8x8，再进行 transpose, reshape 操作。其中 '[' 的部分称为赋值公式，多个公式用 ',' 符号分隔。此外，允许拆分中的某个字符没有赋值，此时会自动推断对应的shape，例如赋值公式只给了 a=2，已知在模型中 ab=32，则自动推断出 b=16；若推断出的shape异常会直接报错，比如 ab=32，若赋值 a=5，则 b=6.4，又维度必须是整数，此时会抛出异常错误。

举例具体说明：[32,4,1,64] ab,c,d,qk[a=2,k=8]->aq,cd,1,kb

第一步：理解左边的“拆分”（把大维度拆成小拼图）

在 `->` 的左边，我们把原本的一个数字，用两个字母拼在一起表示。这代表把这个维度的数值进行了因式分解（拆分）。

- `ab = 32`：意思是把第一个维度 32 拆成了 a 和 b 两部分（即 $a \times b = 32$ ）。
- `c = 4`：第二个维度直接对应 c。
- `d = 1`：第三个维度直接对应 d。
- `qk = 64`：意思是把第四个维度 64 拆成了 q 和 k 两部分（即 $q \times k = 64$ ）。

第二步：理解方括号里的“赋值公式”（给拼图定尺寸）

既然我们知道 `a × b = 32`，那 a 和 b 到底是多少呢？这时候就需要方括号 `[ ]` 里的条件了。

- `[a=2, k=8]`：这是题目给出的已知条件。
 - 因为 `a = 2`，且 `a × b = 32`，所以工具会自动推断出 `b = 16`。
 - 因为 `k = 8`，且 `q × k = 64`，所以工具会自动推断出 `q = 8`。

现在，所有的字母都有了具体的数值：`a=2, b=16, c=4, d=1, q=8, k=8`。

第三步：理解右边的“重组”（按新规则重新拼装）

在 `->` 的右边，就是把这些已经确定好大小的字母，按照新的顺序排列组合起来，形成一个新的张量形状。

- `aq`：变成一个新维度，大小为 $2 \times 8 = 16$ 。
- `cd`：变成一个新维度，大小为 $4 \times 1 = 4$ 。
- `1`：强行插入一个长度为 1 的新维度（扩维）。
- `kb`：变成一个新维度，大小为 $8 \times 16 = 128$ 。

所以，最终输出的形状就变成了：`[16, 4, 1, 128]`。这个过程不仅改变了形状（Reshape），还顺带做了维度的交换（Transpose）。

3. 代码测试

13_onnx_edit

代码块

```
1  from rknn.api import RKNN
2
3  # ===== 第一部分：测试原始的 ONNX 模型 =====
4  # 创建一个 RKNN 对象
5  rknn = RKNN(verbose=True)
6  rknn.config(target_platform='rk3588')
7  rknn.load_onnx("./concat_block.onnx") # 加载当前目录下名为 concat_block.onnx 的原始模型
8  rknn.build(do_quantization=True, dataset='./dataset.txt') # 构建 RKNN 模型:
9  rknn.export_rknn('./concat_block.rknn')
10
11 # ===== 第二部分：使用 rknn.utils.onnx_edit 修改 ONNX 模型 =====
12 # 导入 RKNN 提供的 ONNX 模型编辑工具
13 from rknn.utils import onnx_edit
14 # 调用 onnx_edit 函数对原始模型进行维度变换（通常用于解决维度不匹配或优化内存布局）
15 ret = onnx_edit(model = './concat_block.onnx', # 指定要修改的原始模型路径
16                 export_path = './concat_block_edited.onnx', # 指定修改后新模型的保存路径
17
18                 # 修改输入节点 'k_cache.1' 的维度。'a,b,c,d->1,ad,b,c' 表示将原始的4维张量进行重塑 (Reshape)
19                 inputs_transform = { 'k_cache.1': 'a,b,c,d->1,ad,b,c' },
20                 # 修改输出节点 'k_cache' 的维度。'a,b,c,d->1,ab,c,d' 表示将输出的维度进行合并/变换
21                 outputs_transform = { 'k_cache': 'a,b,c,d->1,ab,c,d' },
22                 dataset = './dataset.txt'
23             )
24
```

```

25 # ===== 第三部分：测试修改 (Edited) 后的 ONNX 模型 =====
26 # 重新创建一个新的 RKNN 对象，准备转换修改后的模型
27 rknn = RKNN(verbose=True)
28 rknn.config(target_platform='rk3588')
29 rknn.load_onnx("./concat_block_edited.onnx") # 加载刚才通过 onnx_edit 生成的、修改
    过维度的新模型
30 # 再次构建 RKNN 模型（开启量化），注意这里使用了专门为新模型准备的数据集文件
31 rknn.build(do_quantization=True,
    dataset='./dataset_for_concat_block_edited.txt')
32 rknn.export_rknn('./concat_block_edited.rknn')
33
34 # ===== 第四部分：对比两者的推理速度和内存占用 =====
35 print("===== 4. 开始对比评估两个 RKNN 模型 =====")
36
37 def evaluate_model(model_path, model_name):
38     rknn = RKNN()
39     print(f"--> 正在加载并评估模型：{model_name}")
40
41     # 1. 加载 RKNN 模型
42     ret = rknn.load_rknn(path=model_path)
43     if ret != 0:
44         print(f'加载 {model_name} 失败!')
45         return
46
47     # 2. 初始化运行时环境
48     # target=None 表示在 PC 上模拟评估。如果你有 RK3588 开发板，可以改成
    target='rk3588' 并插上板子
49     ret = rknn.init_runtime(target='rk3588', perf_debug=True, eval_mem=True)
50     if ret != 0:
51         print('初始化运行时环境失败!')
52         rknn.release()
53         return
54
55     # 3. 评估推理性能 (eval_perf)
56     print(f"\n--- {model_name} 性能评估 (Perf) ---")
57     rknn.eval_perf(is_print=True)
58
59     # 4. 评估内存占用 (eval_memory)
60     print(f"\n--- {model_name} 内存评估 (Memory) ---")
61     rknn.eval_memory(is_print=True)
62
63     rknn.release()
64     print(f"--- {model_name} 评估结束 ---\n")
65
66 # 分别评估两个模型
67 evaluate_model('./concat_block.rknn', '【原始模型 Original】')
68 evaluate_model('./concat_block_edited.rknn', '【修改后模型 Edited】')

```

69

```
70 print("===== 所有评估完成 =====")
```

变换逻辑

```
I rknn building done.
I For 'k_cache.1': 'a,b,c,d->1,ad,b,c'
Input: 'k_cache.1' was reset as shape-[1, 32, 48, 1].(Origin shape is [4, 48, 1, 8])
Insert ops to transform [1, 32, 48, 1] to [4, 48, 1, 8]:
- Insert reshape op. [1, 32, 48, 1] reshape to [4, 8, 48, 1].
- Insert transpose op. [4, 8, 48, 1] transpose(0, 2, 3, 1) to [4, 48, 1, 8].
I For 'k_cache': 'a,b,c,d->1,ab,c,d'
Output: 'k_cache' was reset as shape-[1, 32, 48, 1].(Origin shape is [4, 8, 48, 1])
Insert ops to transform [4, 8, 48, 1] to [1, 32, 48, 1]:
- Insert reshape op. [4, 8, 48, 1] reshape to [1, 32, 48, 1].
I Edited model has been saved to './concat_block_edited.onnx'.
I Dataset file has been assigned. Start transform to get new dataset
100%|████████████████████████████████████████████████████████████████████████████████| 1/1 [00:00<00:00, 1585.75it/s]
I New dataset has been saved to './dataset_for_concat_block_edited.txt'.
I New dataset folder is './dataset_for_concat_block_edited'.
I rknn-toolkit2 version: 2.3.2
I Loading : 100%|████████████████████████████████████████████████████████████████████████████████| 4/4 [00:00<00:00, 44267.06it/s]
W load_onnx: The config.mean_values is None, zeros will be set for input 0!
W load_onnx: The config.std_values is None, ones will be set for input 0!
W load_onnx: The config.mean_values is None, zeros will be set for input 1!
W load_onnx: The config.std_values is None, ones will be set for input 1!
```

内存对比

--- 【原始模型 Original】 内存评估 (Memory) ---	--- 【修改后模型 Edited】 内存评估 (Memory) ---
Memory Profile Info Dump	Memory Profile Info Dump
NPU model memory detail(bytes):	NPU model memory detail(bytes):
Weight Memory: 3.12 KiB	Weight Memory: 3.00 KiB
Internal Tensor Memory: 675.00 KiB	Internal Tensor Memory: 195.00 KiB
Other Memory: 73.88 KiB	Other Memory: 38.00 KiB
Total Memory: 752.00 KiB	Total Memory: 236.00 KiB
INFO: When evaluating memory usage, we need consider the size of model, current model size is: 83.81 KiB	INFO: When evaluating memory usage, we need consider the size of model, current model size is: 45.62 KiB
--- 【原始模型 Original】 评估结束 ---	--- 【修改后模型 Edited】 评估结束 ---
I rknn-toolkit2 version: 2.3.2	

性能对比

--- 【原始模型 Original】 性能评估 (Perf) ---	--- 【修改后模型 Edited】 性能评估 (Perf) ---
CPU Current Frequency List:	CPU Current Frequency List:
- 1800000	- 1800000
- 2400000	- 2400000
- 2400000	- 2400000
NPU Current Frequency List:	NPU Current Frequency List:
- 1000000000	- 1000000000
DDR Current Frequency List:	DDR Current Frequency List:
- 1596000000	- 1596000000
Total Operator Elapsed Per Frame Time(us): 1040	Total Operator Elapsed Per Frame Time(us): 291
Total Memory Read/Write Per Frame Size(KB): 774.80	Total Memory Read/Write Per Frame Size(KB): 293.23